

Part 1: Business Requirements Document (BRD)

1. Executive Summary

The **Multi-Property Rental Management System** is designed to provide landlords and property managers with a centralized, automated platform to manage a diverse portfolio of independent real estate assets. The goal is to eliminate manual bookkeeping, reduce rent arrears through automated triggers, and provide clear financial visibility per property.

2. Business Objectives

- **Scalability:** Enable a single user to manage multiple, independent properties with zero data crossover.
- **Efficiency:** Automate 90% of rent collection reconciliation via direct API integration.
- **Transparency:** Provide tenants and landlords with a "Single Source of Truth" regarding financial statements and deposits.
- **Risk Mitigation:** Ensure all legal and personal data (Next of Kin, ID numbers) is captured and retrievable for every lease.

3. Stakeholders

- **Landlord/Admin:** Portfolio owner requiring high-level financial health stats.
- **Property Manager:** Operations user managing daily tasks, tenant onboarding, and expenses.
- **Tenant:** The end-user paying rent and requiring clear communication/receipts.
- **Finance Team:** Users responsible for tax reporting and deposit refunds.

4. High-Level Business Requirements (Functional)

ID	Requirement	Description
BR-01	Multi-Property Support	Each property must have its own independent ledger, units, and settings.
BR-02	Automated Billing	The system must generate monthly invoices automatically.
BR-03	Integrated Payments	Real-time reconciliation via payment gateway APIs (e.g., M-Pesa, Stripe).

BR-04	Communication	Automated SMS/Email for reminders, receipts, and news.

BR-05 Expense Tracking Full capture of property costs to calculate Net Operating Income (NOI).

Part 2: Software Requirements Specification (SRS)

1. System Architecture

The system follows a **Cloud-Native Multi-Tenant Architecture**.

- **Backend:** RESTful API (Node.js/Python) with a PostgreSQL database.
- **Frontend:** React/Next.js Web Dashboard (Responsive).
- **Integrations:** WhatsApp, SMS Gateways (Twilio/AfricasTalking), Payment Gateways, Cloud Storage (AWS S3) for documents.

2. Data Model & Schema Design

The database must enforce isolation using `property_id` and `organization_id`.

2.1 Entity Relationship Diagram (Conceptual)

- **Organization 1:N Properties**
- **Property 1:N Units**
- **Unit 1:1 Active Lease**
- **Lease 1:1 Tenant**
- **Tenant 1:N Transactions**

2.2 Key Table Specifications

- **Tenant Profile:** Name, ID Number, Phone, Email, Primary Source of Income, NOK Details (Name, Relationship, Phone).
- **Lease:** Start/End Date, Rent Amount, Security Deposit, Status (Active/Terminated).
- **Transactions:** PropertyID, TenantID, Type (Debit/Credit/Refund), Reference, Timestamp, Metadata (JSON).

3. Functional Requirements

3.1 Tenant & Lease Management

- **Onboarding:** Create tenant profiles with mandatory Next of Kin and ID verification.
- **Unit Assignment:** Map tenants to specific houses within a selected property.
- **Deposit Management:** System must "lock" deposit funds, preventing them from being counted as revenue until the lease is terminated and deductions (if any) are applied.

3.2 The Financial Engine

- **Monthly Invoicing:** A cron job runs on the 1st of each month to post rent debits to tenant statements.
- **API Reconciliation:**
 1. Webhook receives payment.
 2. Logic matches AccountReference to TenantID.
 3. If match found, auto-credit account and trigger SMS receipt.
 4. If no match, move to **Suspense Account** for manual landlord mapping.
- **Batch Uploads:** Parser for CSV/Excel bank statements to reconcile offline payments.

3.3 Dashboard & Reporting

- **Global View:** Aggregated stats (Total Portfolio Revenue, Total Arrears).
- **Property View:** Filtered metrics (Occupancy %, Property-specific P&L).
- **Tenant Statement:** A downloadable PDF ledger showing every debit and credit since lease inception.

4. Non-Functional Requirements

4.1 Security

- **Role-Based Access Control (RBAC):** Property managers can only see properties assigned to them.
- **Data Encryption:** Encrypt PII (ID numbers, Phone numbers) at rest.
- **Audit Logs:** Record every manual change to a transaction or rent amount (Who, When, Old Value, New Value).

4.2 Performance & Availability

- **Latency:** Dashboard widgets must load in < 1.5s.
- **Scalability:** Database design must support 10,000+ units across 500+ properties without performance degradation.

5. User Interface (UX) Requirements

- **Context Switcher:** A persistent dropdown to switch between "All Properties" and specific buildings.
- **Search:** Global search across Name, ID Number, House Number, or Phone Number. ●
- **Mobile-Ready:** Tenants must be able to view their statements clearly on mobile browsers.

6. System State Machine (Deposit Refund)

To ensure financial integrity, developers must follow this logic flow:

1. **Status: Active** (Deposit held in trust).
2. **Status: Notice Given** (Lease set to expire).
3. **Status: Inspection** (Landlord enters damage costs).
4. **Status: Final Settlement** (System calculates: $\text{Refund} = \text{Deposit} - \text{Damages} - \text{Arrears}$).
5. **Status: Closed** (Refund issued, unit marked Vacant).